

# Covid\_19\_Final\_Project

Aaron\_Warsaw

2026-09-14

## R Markdown

```
knitr::opts_chunk$set(echo = TRUE)
library(tidyverse)
library(lubridate)
library(dplyr)
```

## Getting Covid 19 Data

Begin by reading in 4 data sets from John Hopkins Covid 19 Github website. The data is in csv format. There are two data sets with United States Covid data and two websites with global covid data.

I am interested in looking at covid cases vs covid deaths. Can the number of covid cases be used as a predictor of the number of covid deaths in the United States and in my home state of Missouri as well.

```
global_cases <- read.csv(
  "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_time_series/global_cases_time_series.csv"
)

global_deaths <- read.csv(
  "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_time_series/global_deaths_time_series.csv"
)

us_cases <- read.csv(
  "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_time_series/us_cases_time_series.csv"
)

us_deaths <- read.csv(
  "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_time_series/us_deaths_time_series.csv"
)
```

Checking to see if data was properly read in.

```
head(global_cases)
head(global_deaths)
head(us_cases)
head(us_deaths)
```

Data looks good for all four csv files.

# Cleaning and organizing data

## Cleaning and organizing Global data sets.

I don't need lat/long for my analysis. Each state or country is a row and each date is a column. Will put date, cases, deaths in their own column. Rename Region and State to be more R friendly with and "/" between them instead of a "."

Tidy global\_cases

```
global_cases <- global_cases %>%
  pivot_longer(
    cols = -c('Province.State', 'Country.Region', Lat, Long),
    names_to = "date",
    values_to = "cases"
  ) %>%
  select(-Lat, -Long)

global_deaths <- global_deaths %>%
  pivot_longer(
    cols = -c('Province.State', 'Country.Region', Lat, Long),
    names_to = "date",
    values_to = "deaths"
  ) %>%
  select(-Lat, -Long)
```

Checking global\_cases and global\_deaths to see if they are formatted the way I want now. Checking to see if data was properly read in.

```
head(global_cases)
head(global_deaths)
```

Data looks good, but we need to change the date format into a date variable. I will do that in the next step as well as combine the global\_cases and global\_deaths data sets into one called global.

```
names(global_cases)
```

```
## [1] "Province.State" "Country.Region" "date"          "cases"
```

```
names(global_deaths)
```

```
## [1] "Province.State" "Country.Region" "date"          "deaths"
```

Joining global\_cases with global deaths. Also changing the date format by getting rid of the "X" in front of the date and replacing periods with /'s.

```
global <- global_cases %>%
  full_join(
    global_deaths,
    by = c("Province.State", "Country.Region", "date")
  ) %>%
```

```

rename(
  `Country_Region` = Country.Region,
  `Province_State` = Province.State
) %>%
mutate(date = mdy(gsub("\\.", "/", sub("^X", "", date))))

```

Checking new global data set.

```
head(global)
```

Data format looks good. Now to look at summary stats for global data set.

```
summary(global)
```

```

##      Province_State      Country_Region      date      cases
## Length :330327      Length :330327      Min. :2020-01-22      Min. :      0
## N.unique : 92      N.unique : 201      1st Qu.:2020-11-02      1st Qu.:     680
## N.blank :226314      N.blank : 0      Median :2021-08-15      Median :    14429
## Min.nchar: 0      Min.nchar: 2      Mean :2021-08-15      Mean :   959384
## Max.nchar: 44      Max.nchar: 32      3rd Qu.:2022-05-28      3rd Qu.:  228517
##                                     Max. :2023-03-09      Max. :103802702
##
##      deaths
## Min. : 0
## 1st Qu.: 3
## Median : 150
## Mean : 13380
## 3rd Qu.: 3032
## Max. :1123836

```

Looks like a large number of dates have zero cases. Lets find how many rows have cases = 0

```
sum(global$cases == 0, na.rm = TRUE)
```

```
## [1] 23500
```

This is a lot, 23500 rows have cases = 0. Filter out rows where cases = 0

```

global <- global %>% filter(cases > 0)
summary(global)

```

```

##      Province_State      Country_Region      date      cases
## Length :306827      Length :306827      Min. :2020-01-22      Min. :      1
## N.unique : 91      N.unique : 201      1st Qu.:2020-12-12      1st Qu.:    1316
## N.blank :210689      N.blank : 0      Median :2021-09-16      Median :    20365
## Min.nchar: 0      Min.nchar: 2      Mean :2021-09-11      Mean :  1032863
## Max.nchar: 44      Max.nchar: 32      3rd Qu.:2022-06-15      3rd Qu.:  271281
##                                     Max. :2023-03-09      Max. :103802702
##
##      deaths
## Min. : 0
## 1st Qu.: 7

```

```
## Median :    214
## Mean   :   14405
## 3rd Qu.:    3665
## Max.   : 1123836
```

We can see the min cases number is 1. So we successfully removed all rows where cases = 0.

Maximum number of cases looks high. Lets check if it looks like a mistake.

```
global %>% filter(cases > 90000000)
```

Ok so we have numerous dates with over 90 million cases so our maximum number of cases of 103 million seems correct.

## Cleaning and Organizing United States Data Sets.

Checking column names and content of US data sets.

```
head(us_cases)
head(us_deaths)
names(us_cases)
names(us_deaths)
```

Data looks good. Now I will work with US\_cases data set. Start by pivoting so that each date is a row, change date data type into a date data type, and remove Lat and Long\_ columns.

```
us_cases <- us_cases %>%
  pivot_longer(cols = -(UID:Combined_Key),
               names_to = "date",
               values_to = "cases") %>%
  select(Admin2:cases) %>%
  mutate(date = mdy(gsub("\\.", "/", sub("^X", "", date)))) %>%
  select(-Lat, -Long_)
head(us_cases)
```

US\_cases now has the following columns: Admin2, Province\_State, Country\_Regions, Combined\_Key, date, cases.

Now to tidy the us\_deaths dataset. Looking above the column names are the same except that the us\_deaths data set has a Population column before the dates.

Selecting columns to use, pivoting so each date column is now a row, and fixing date data type into date data type.

```
us_deaths <- us_deaths %>%
  pivot_longer(cols = -(UID:Population),
               names_to = "date",
               values_to = "deaths") %>%
  select(Admin2:deaths) %>%
  mutate(date = mdy(gsub("\\.", "/", sub("^X", "", date)))) %>%
  select(-Lat, -Long_)
us_deaths
```

Joining us\_cases and us\_deaths.

```
united_states <- us_cases %>%
  full_join(us_deaths,
    by = c("Admin2", "Province_State", "Country_Region", "Combined_Key", "date"))
```

Checking new United States data set.

```
head(united_states)
names(global)
```

Joined united\_states data set looks good.

## Adding Population to Global Data Set.

However, we do not yet have population for the global data set. Adding it below.

```
global <- global %>%
  unite("Combined_Key",
    c(`Province_State`, `Country_Region`),
    sep = ", ",
    na.rm = TRUE,
    remove = FALSE)
global
```

Reading in the file from John Hopkins Github Website. A file with population for countries.

```
population <- read_csv("https://raw.githubusercontent.com/CSSEGISandData/COVID-19/refs/heads/master/csse_covid_19_data/csse_covid_19_time_series/time_series_covid19_population_by_country.csv")
select(-c(Lat, Long_, Combined_Key, code3, iso2, iso3, Admin2))
```

```
## Rows: 4321 Columns: 12
## -- Column specification -----
## Delimiter: ","
## chr (7): iso2, iso3, FIPS, Admin2, Province_State, Country_Region, Combined_Key
## dbl (5): UID, code3, Lat, Long_, Population
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
population
```

Adding population to the global data set.

```
global <- global %>%
  left_join(population, by = c("Country_Region")) %>%
  select(-c(UID, FIPS)) %>%
  select(Country_Region, date, cases, deaths, Population, Combined_Key)
```

```
## Warning in left_join(., population, by = c("Country_Region")): Detected an unexpected many-to-many relationship.
## i Row 9253 of 'x' matches multiple rows in 'y'.
## i Row 1 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship = "many-to-many"' to silence this warning.
```

```
global
```

## Data Visualization.

### United States by State Summaries.

```
US_by_state <- united_states %>%  
  group_by(Province_State, Country_Region, date) %>%  
  summarize(  
    cases = sum(cases, na.rm = TRUE),  
    deaths = sum(deaths, na.rm = TRUE),  
    Population = sum(Population, na.rm = TRUE),  
    .groups = "drop"  
  ) %>%  
  mutate(  
    deaths_per_million = deaths * 1000000 / Population  
  )
```

Summary of each state by date.

```
US_by_state
```

Checking the end of the US\_by\_State data set.

```
tail(US_by_state)
```

OK, we have all the states.

Now look at US totals data set.

```
US_totals <- US_by_state %>%  
  group_by(Country_Region, date) %>%  
  summarize(  
    cases = sum(cases, na.rm = TRUE),  
    deaths = sum(deaths, na.rm = TRUE),  
    Population = sum(Population, na.rm = TRUE),  
    .groups = "drop"  
  ) %>%  
  mutate(  
    deaths_per_million = deaths * 1000000 / Population  
  ) %>%  
  select(  
    Country_Region, date, cases, deaths,  
    deaths_per_million, Population  
  )
```

```
US_totals
```

Checking the end of the US\_totals data set.

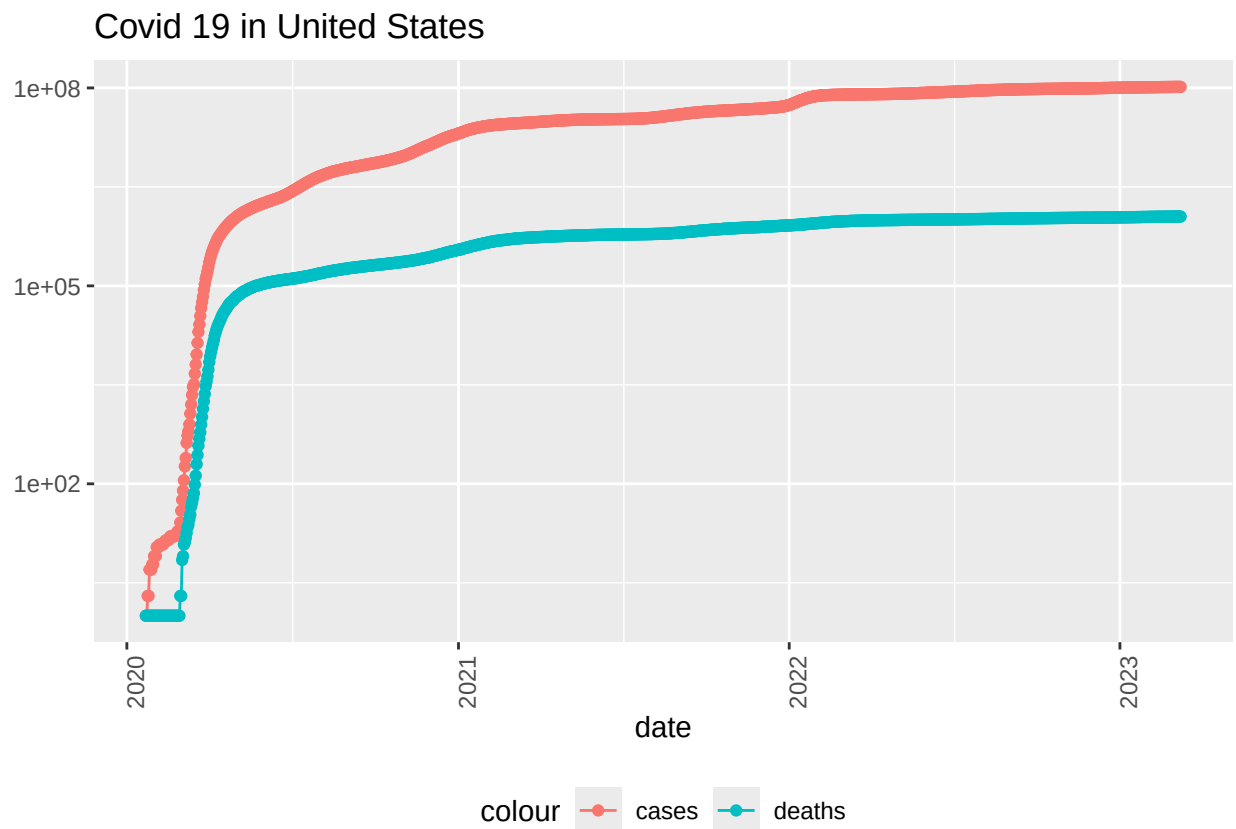
```
tail(US_totals)
```

Data looks good.

## Visualization 1

Graph of United States Covid 19 cases and deaths.

```
US_totals %>%  
  filter(cases > 0) %>%  
  ggplot(aes(x = date, y = cases)) +  
  geom_line(aes(color = "cases")) +  
  geom_point(aes(color = "cases")) +  
  geom_line(aes(y = deaths, color = "deaths")) +  
  geom_point(aes(y = deaths, color = "deaths")) +  
  scale_y_log10() +  
  theme(legend.position = "bottom",  
        axis.text.x = element_text(angle = 90)) +  
  labs(title = "Covid 19 in United States", y = NULL)
```



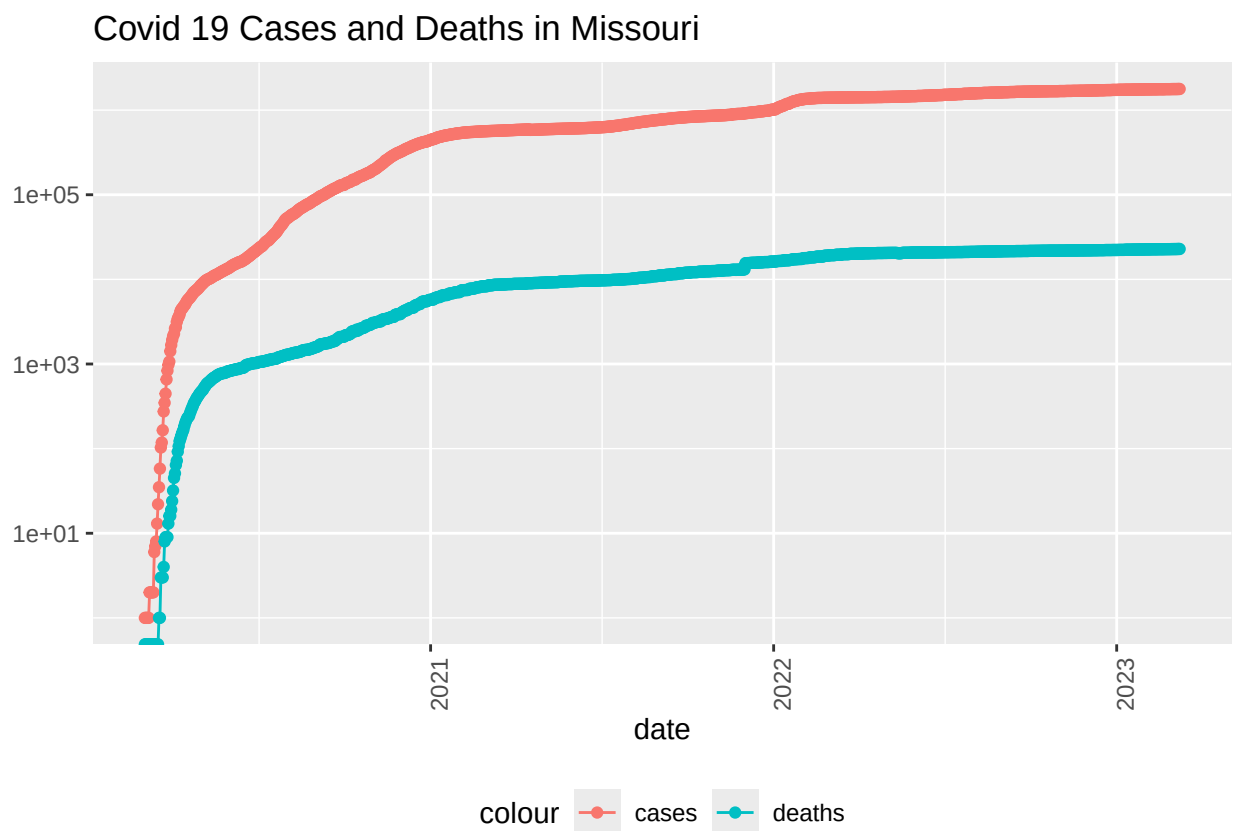
## Visualization 2

Graph of Missouri Covid 19 cases and deaths.

```

state <- "Missouri"
US_by_state %>%
  filter(Province_State == state) %>%
  filter(cases > 0) %>%
  ggplot(aes(x = date, y = cases)) +
  geom_line(aes(color = "cases")) +
  geom_point(aes(color = "cases")) +
  geom_line(aes(y = deaths, color = "deaths")) +
  geom_point(aes(y = deaths, color = "deaths")) +
  scale_y_log10() +
  theme(legend.position = "bottom",
        axis.text.x = element_text(angle = 90)) +
  labs(title = str_c("Covid 19 Cases and Deaths in ", state), y = NULL)

```



Getting max date and deaths for United States and Missouri.

```

US_max_date <- max(US_totals$date)
US_max_deaths <- max(US_totals$deaths)

US_max_date

```

```
## [1] "2023-03-09"
```

```
US_max_deaths
```

```
## [1] 1123836
```

## Analyzing Data

It looks like the number of cases and deaths taper off and almost flatlines. Is this true or not? I will investigate further.

Adding new variable and transform data again. Adding new\_cases variable which is cases - lag(cases). Adding new\_deaths variable which is deaths - lag(deaths).

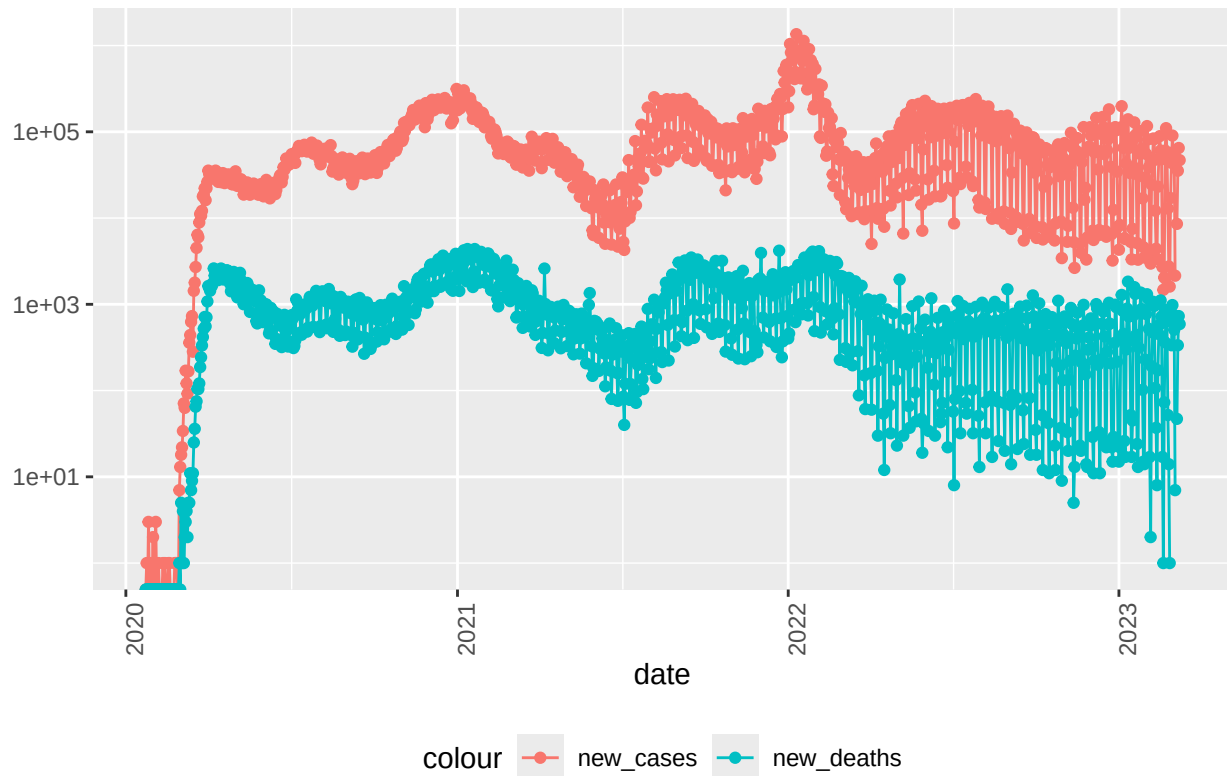
```
US_by_state <- US_by_state %>%  
  mutate(new_cases = cases - lag(cases),  
         new_deaths = deaths - lag(deaths))  
US_totals <- US_totals %>%  
  mutate(new_cases = cases - lag(cases),  
         new_deaths = deaths - lag(deaths))  
tail(US_totals %>%  
  select(new_cases, new_deaths, everything()))
```

## Visualization 3

Now we can graph new cases and new deaths.

```
US_totals %>%  
  ggplot(aes(x = date, y = new_cases)) +  
  geom_line(aes(color = "new_cases")) +  
  geom_point(aes(color = "new_cases")) +  
  geom_line(aes(y = new_deaths, color = "new_deaths")) +  
  geom_point(aes(y = new_deaths, color = "new_deaths")) +  
  scale_y_log10() +  
  theme(legend.position = "bottom",  
        axis.text.x = element_text(angle = 90)) +  
  labs(title = "Covid 19 New Cases and Deaths in United States", y = NULL)
```

## Covid 19 New Cases and Deaths in United States



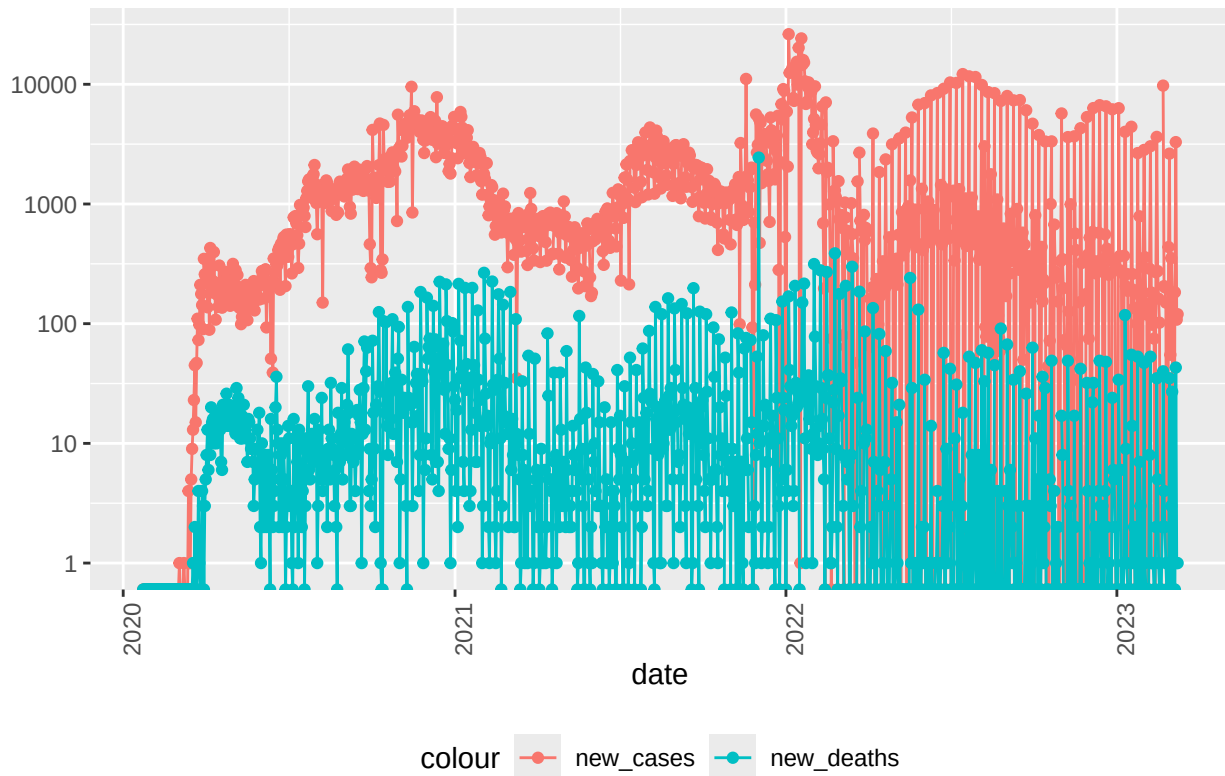
Now we can see the up and down trends in new cases and new deaths in more detail although the overall trend does look like it has flattened out. However, the number of new cases and new deaths remain high.

## Visualization 4

Now let's look at the same graph for Missouri.

```
state <- "Missouri"
US_by_state %>%
  filter(Province_State == state) %>%
  ggplot(aes(x = date, y = new_cases)) +
  geom_line(aes(color = "new_cases")) +
  geom_point(aes(color = "new_cases")) +
  geom_line(aes(y = new_deaths, color = "new_deaths")) +
  geom_point(aes(y = new_deaths, color = "new_deaths")) +
  scale_y_log10() +
  theme(legend.position = "bottom",
        axis.text.x = element_text(angle = 90)) +
  labs(title = str_c("New Covid 19 Cases and Deaths in ", state), y = NULL)
```

## New Covid 19 Cases and Deaths in Missouri



Missouri has a similar pattern of new covid 19 cases and new deaths to the United States totals. There is a dramatic increase in new cases and deaths at the end of 2021 followed by a dramatic decrease in new cases and deaths at the beginning of 2022. After the decrease in cases and deaths at the beginning of 2022 the cases gradually rise again and seem to level off. However, the number of new cases and new deaths remain high.

## Modeling Data United States

Lets look at a linear model of deaths\_per\_thousand being a function of cases\_per\_thousand in the US\_state\_totals data set.

Making US\_state\_totals data set to use with linear model.

```
US_state_totals <- US_by_state %>%
  group_by(Province_State) %>%
  summarize(deaths = max(deaths), cases = max(cases),
            population = max(Population),
            cases_per_thou = 1000 * cases / population,
            deaths_per_thou = 1000 * deaths / population) %>%
  filter(cases > 0, population > 0)
```

Now running the linear model of deaths per thousand as a function of cases per thousand.

```
linear_model <- lm(deaths_per_thou ~ cases_per_thou, data = US_state_totals)
summary(linear_model)
```

```
##
## Call:
## lm(formula = deaths_per_thou ~ cases_per_thou, data = US_state_totals)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.3352 -0.5978  0.1491  0.6535  1.2086
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -0.36167    0.72480  -0.499    0.62
## cases_per_thou  0.01133    0.00232   4.881 9.76e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8615 on 54 degrees of freedom
## Multiple R-squared:  0.3061, Adjusted R-squared:  0.2933
## F-statistic: 23.82 on 1 and 54 DF,  p-value: 9.763e-06
```

The fitted model is:  $\text{deaths\_per\_thousand} = -0.36167 + 0.01133 \times \text{cases\_per\_thousand}$ . There was a statistically significant positive association between cases per thousand and deaths per thousand. Each additional case per thousand associated with an estimated 0.0113 increase in deaths per thousand ( $p < 0.001$ ). The Adjusted R-squared value of 0.2933 explains 0.2933% of the variation in deaths per thousand.

Find the number of cases that are the smallest.

```
US_state_totals %>% slice_min(cases_per_thou)
```

Find the number of cases that are the largest.

```
US_state_totals %>% slice_max(cases_per_thou)
```

Cases per thousand go from a minimum of 55641 in American Samoa to a maximum of 460697 in Rhode Island.

Adding prediction column based on the linear model above to the data set.

```
US_state_totals %>%
  mutate(pred = predict(linear_model))
```

I am now interested in seeing how well the model predicts the actual values in the dataset.

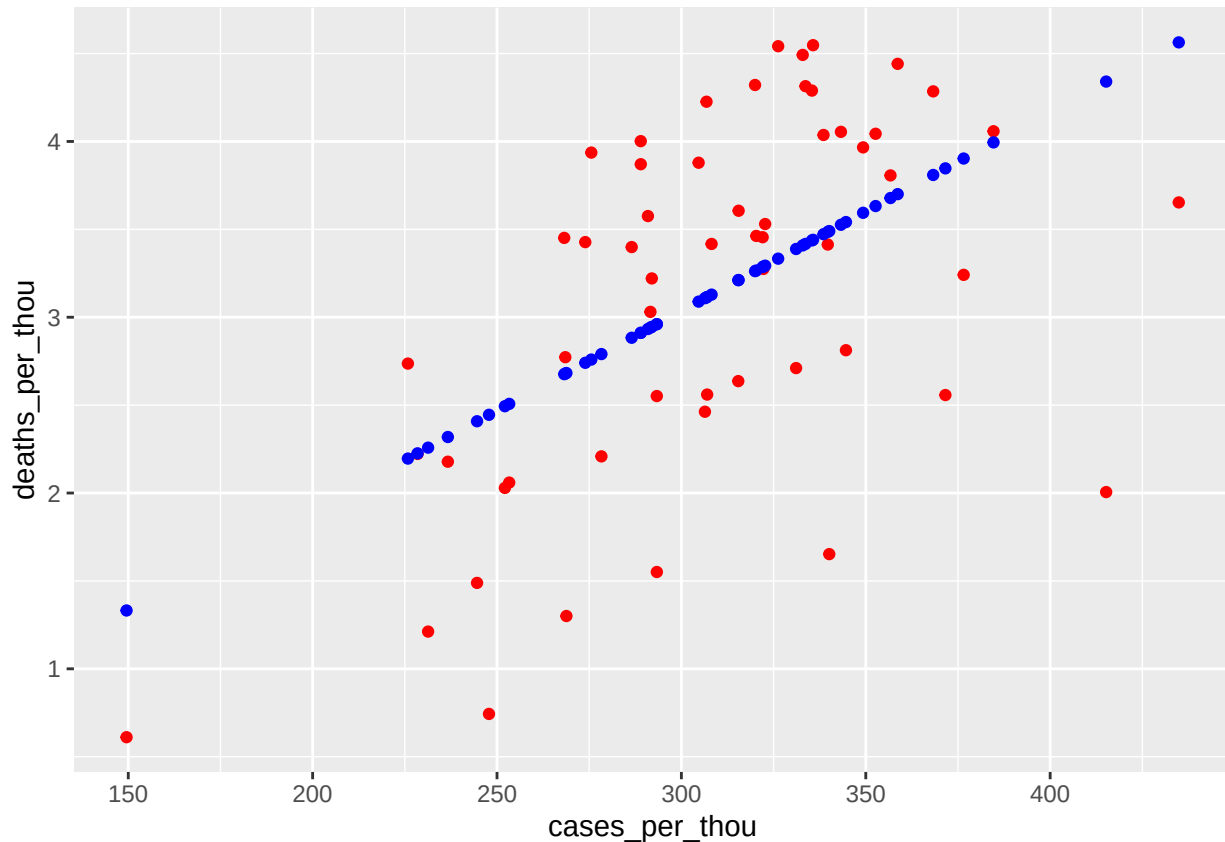
Make new dataset US totals with predictions.

```
US_totals_w_predictions <- US_state_totals %>%
  mutate(pred = predict(linear_model))
US_totals_w_predictions
```

## Model Visualization: United States

Plotting actual deaths per thousand vs predicted deaths per thousand.

```
US_totals_w_predictions %>%
  ggplot() +
  geom_point(aes(x = cases_per_thou, y = deaths_per_thou),
             color = "red") +
  geom_point(aes(x = cases_per_thou, y = pred), color = "blue")
```



Actual values of cases per thousand as a predictor of deaths per thousand (red dots) show a weak linear appearance with a similar deviation from the linear predicted values (blue dots). So although the model has a significant positive association between actual and predicted values the adjusted R-squared of 0.29 leaves room for much improvement in future model development.

Now let's look at modeling cases per thousand as a predictor of deaths per thousand for the rest of the world, excluding the United States.

## Modeling Data: All countries except United States

Global cases per thousand as a predictor of global deaths per thousand.

```
head(global)
```

```
## # A tibble: 6 x 6
##   Country_Region date       cases deaths Population Combined_Key
##   <chr>          <date>   <int> <int>    <dbl> <chr>
## 1 Afghanistan  2020-02-24     5     0  38928341 , Afghanistan
## 2 Afghanistan  2020-02-25     5     0  38928341 , Afghanistan
```

```
## 3 Afghanistan    2020-02-26     5     0 38928341 , Afghanistan
## 4 Afghanistan    2020-02-27     5     0 38928341 , Afghanistan
## 5 Afghanistan    2020-02-28     5     0 38928341 , Afghanistan
## 6 Afghanistan    2020-02-29     5     0 38928341 , Afghanistan
```

```
tail(global)
```

```
## # A tibble: 6 x 6
##   Country_Region date       cases deaths Population Combined_Key
##   <chr>          <date>    <int> <int>      <dbl> <chr>
## 1 Zimbabwe      2023-03-04 264127  5668  14862927 , Zimbabwe
## 2 Zimbabwe      2023-03-05 264127  5668  14862927 , Zimbabwe
## 3 Zimbabwe      2023-03-06 264127  5668  14862927 , Zimbabwe
## 4 Zimbabwe      2023-03-07 264127  5668  14862927 , Zimbabwe
## 5 Zimbabwe      2023-03-08 264276  5671  14862927 , Zimbabwe
## 6 Zimbabwe      2023-03-09 264276  5671  14862927 , Zimbabwe
```

Remove rows with NAN values

```
#Global_by_country <- na.omit(global)
```

```
US_by_state <- united_states %>%
  group_by(Province_State, Country_Region, date) %>%
  summarize(
    cases = sum(cases, na.rm = TRUE),
    deaths = sum(deaths, na.rm = TRUE),
    Population = sum(Population, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    deaths_per_million = deaths * 1000000 / Population
  )
```

Summary of each state by date.

```
US_by_state
```

```
head(global, 100)
```

Columns I will need: Country\_Region, cases, date, deaths, Population Global data by country

```
Global_by_country <- global %>%
  group_by(Country_Region, date) %>%
  summarize(
    cases = sum(cases, na.rm = TRUE),
    deaths = sum(deaths, na.rm = TRUE),
    Population = sum(Population, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    deaths_per_million = deaths * 1000000 / Population
  )
```

Summary of each country by date.

```
Global_by_country
```

Lets look at a linear model of deaths\_per\_thousand being a function of cases\_per\_thousand in the US\_state\_totals data set.

Making Global\_country\_totals data set to use with linear model.

```
Global_country_totals <- Global_by_country %>%
  group_by(Country_Region) %>%
  summarize(deaths = max(deaths), cases = max(cases),
            population = max(Population),
            cases_per_thou = 1000 * cases / population,
            deaths_per_thou = 1000 * deaths / population) %>%
  filter(cases >=0, population >= 0)
Global_country_totals
```

Checking for NAN's in rows.

```
sum(apply(Global_country_totals, 1, anyNA))
```

```
## [1] 3
```

Removing rows with nan

```
Global_country_totals_cleaned <- Global_country_totals[
  is.finite(Global_country_totals$deaths_per_thou) &
  is.finite(Global_country_totals$cases_per_thou),
]
Global_country_totals_cleaned
```

Now removing US because I am interested in the rest of the world.

Getting country names to drop the U.S.

```
unique(Global_country_totals_cleaned$Country_Region)
```

United states is labeled "US"

```
Global_country_totals_cleaned_noUS <- Global_country_totals_cleaned[
  Global_country_totals_cleaned$Country_Region != "US",
]
```

Now running the linear model of deaths per thousand as a function of cases per thousand.

```
linear_model_global <- lm(
  deaths_per_thou ~ cases_per_thou,
  data = Global_country_totals_cleaned_noUS
)
summary(linear_model_global)
```

The fitted model is:  $\text{deaths\_per\_thousand} = -0.9555 + 0.0114 \times \text{cases\_per\_thousand}$ . There was statistically significant positive association between cases per thousand and deaths per thousand. Each additional case per thousand associated with an estimated 0.0108271 increase in deaths per thousand ( $p < 0.001$ ). The adjusted R-squared value of 0.5923 explains 59.2% of the variation in deaths per thousand. This almost a perfect fit for cases per thousand predicting deaths per thousand.

Now we want to look at predicted vs actual values. Adding prediction column based on the linear model above to the data set.

```
#Global_country_totals_cleaned <- na.omit(Global_country_totals)
Global_country_totals_cleaned_noUS %>%
  mutate(pred = predict(linear_model_global))
```

I am now interested in seeing how well the model predicts the actual values in the dataset.

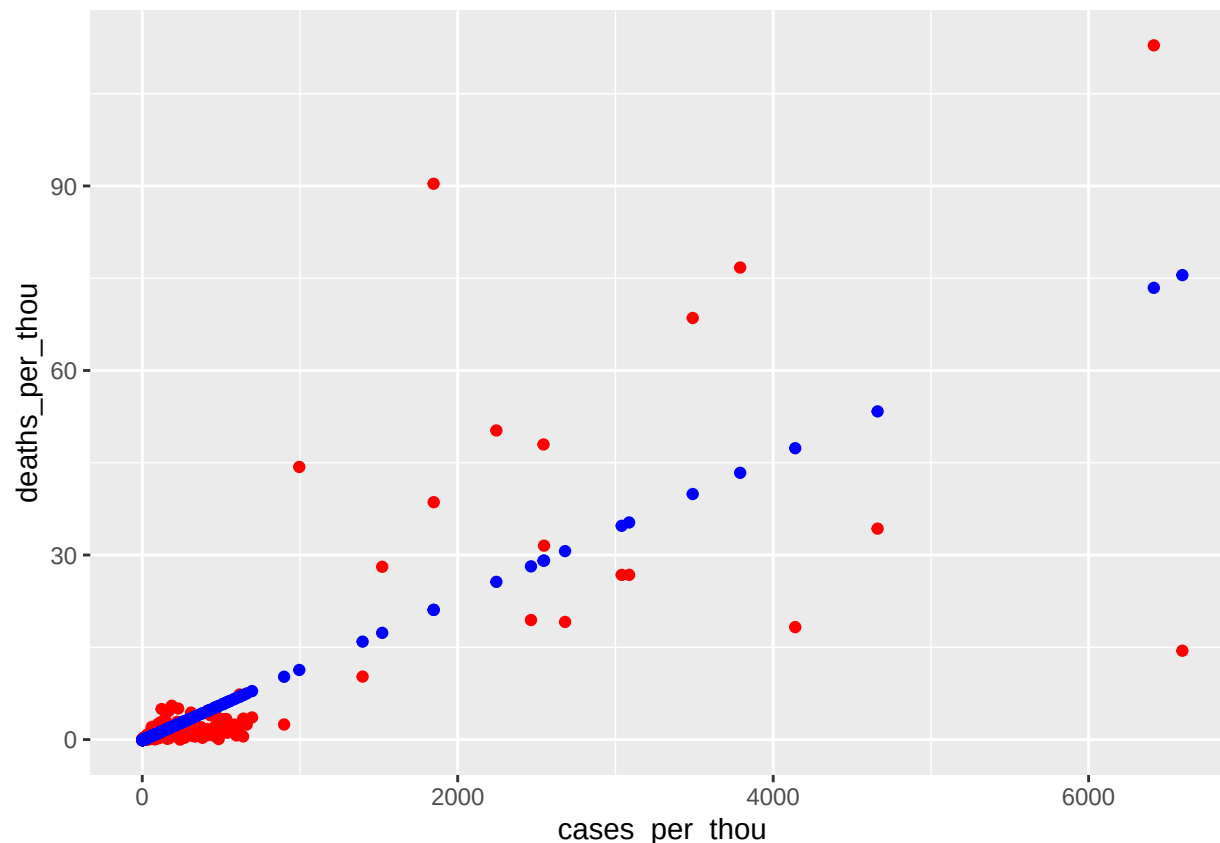
Make new dataset global totals with predictions.

```
Global_totals_w_predictions <- Global_country_totals_cleaned_noUS %>%
  mutate(pred = predict(linear_model_global))
Global_totals_w_predictions
```

## Model Visualization: All Countries except United States

Plotting actual deaths per thousand vs predicted deaths per thousand.

```
Global_totals_w_predictions %>%
  ggplot() +
  geom_point(aes(x = cases_per_thou, y = deaths_per_thou),
             color = "red") +
  geom_point(aes(x = cases_per_thou, y = pred), color = "blue")
```



The predicted vs actual values of cases per thousand as a as a predictor of deaths per thousand are heavily concentrated on the lower left side of the graph but become fewer and farther apart from predicted values once deaths per thousand hit 1000. This suggests the model will be more accurate for cases per thousand numbers below 1000. That can be tested in further analysis.

## Conclusion

Just like the model for the United States the model for the rest of the world had a significant positive linear relationship between cases per thousand and deaths per thousand. The adjusted R-squared value of 0.59 for the rest of the world was 0.59 compared to the R-squared value of the United States (0.29). However further model development and consideration of other predictive variables should be investigated for both models.

Bias in the data could come from the ways in which deaths are reported. For example, do they differ among states and among countries? Any differences in how deaths due to Covid 19 are classified and reported could make a difference in the models ability to correctly predict deaths per thousand. Also, how quickly the deaths per day are reported. Are they all reported the same day or does reporting time lag in less developed countries.

## Session Information

```
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24 ucrt)
```

```

## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/Chicago
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] lubridate_1.9.5 forcats_1.0.1  stringr_1.6.0  dplyr_1.2.1
## [5] purrr_1.2.2     readr_2.2.0    tidyr_1.3.2    tibble_3.3.1
## [9] ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] bit_4.6.0          gtable_0.3.6       crayon_1.5.3       compiler_4.6.1
## [5] tidyselect_1.2.1   parallel_4.6.1     scales_1.4.0       yaml_2.3.12
## [9] fastmap_1.2.0      R6_2.6.1           labeling_0.4.3     generics_0.1.4
## [13] curl_7.1.0         knitr_1.51         pillar_1.11.1      RColorBrewer_1.1-3
## [17] tzdb_0.5.0         rlang_1.3.0        utf8_1.2.6         stringi_1.8.9
## [21] xfun_0.60          S7_0.2.2           bit64_4.8.2        otel_0.2.0
## [25] timechange_0.4.0   cli_3.6.6          withr_3.0.3        magrittr_2.0.5
## [29] digest_0.6.39      grid_4.6.1         vroom_1.7.1        rstudioapi_0.19.0
## [33] hms_1.1.4          lifecycle_1.0.5    vctrs_0.7.3        evaluate_1.0.5
## [37] glue_1.8.1         farver_2.1.2       rmarkdown_2.31     tools_4.6.1
## [41] pkgconfig_2.0.3    htmltools_0.5.9

```